1. Compare raise B from A, raise B inside an except block, and raise B from None. What does the traceback show in each case?

- raise B from A shows that error B happened because of error A. The traceback shows both errors and their connection.

- raise B inside an except block also shows the original error, but it is shown as the context of the new error.

- raise B from None shows only error B. It hides the original error from the traceback.

2. Why is collecting errors into an ExceptionGroup better for this system than raising the first error found?

It is better because the program can show all the errors at once instead of stopping at the first one. This helps the user see and fix all the problems without running the program many times.

3. What would happen if EnrollmentTransaction.**__exit__** returned True?

If it returns True, the error will be hidden, and the program will continue running. This can be a problem because the user might not know that the enrollment failed.

4. Why must the retry decorator not retry EnrollmentError? Give a concrete harmful example.

The retry decorator should not retry EnrollmentError because it is a problem with the enrollment itself, not just a temporary error. For example, if a student is already enrolled in a course, retrying the same enrollment will not fix the problem. It may also cause duplicate records if the system is not properly protected.

5. In Test 9, trace step by step how the rollback happened even though the error came from inside a decorated function.

1. The program starts the enrollment transaction.

2. The decorated function runs inside the transaction.

3. The function tries to enroll the student but an error happens.

4. The decorator does not retry the EnrollmentError.

5. The error goes back to the transaction block.

6. The **__exit__** method detects the error and performs the rollback.

7. The changes made during the transaction are undone, and the error is still shown.